

Baseline:



```
public void readFrom(final InputStream in) throws IOException {
    buffer.clear();
    int read;
    while ((read = in.read(buffer.array(), buffer.position(), buffer.remaining())) != -1) {
        buffer.position(buffer.position() + read);
    }
    buffer.flip();
}
```



Structure-Constrained:

```
public void readFrom(final InputStream in) throws IOException {
    buffer.clear();
    int read;
    while ((read = in.read(buffer.array(), buffer.position(), buffer.remaining())) != -1) {
        buffer.position(buffer.position() + read);
    }
    buffer.flip();
}
```



Robust-Handling:

```
public void readFrom(final InputStream in) throws IOException {
    if (in == null) {
        throw new IllegalArgumentException("InputStream cannot be null");
    }
    try {
        byte[] buffer = new byte[in.available()];
        int bytesRead = in.read(buffer);
        if (bytesRead != buffer.length) {
            throw new IOException("Failed to read the expected number of bytes");
        }
        this.buffer = buffer;
        this.position = 0;
    } finally {
        in.close();
    }
}
```



Reasoning-Guided:

```
public void readFrom(final InputStream in) throws IOException {
    if (in == null) {
        throw new IllegalArgumentException("InputStream cannot be null");
    }
    try {
        byte[] buffer = new byte[in.available()];
        int bytesRead = in.read(buffer);
        if (bytesRead != buffer.length) {
            throw new IOException("Failed to read the expected number of bytes");
        }
        this.buffer = buffer;
    } finally {
        in.close();
    }
}
```



Edge-Coverage:

```
public void readFrom(final InputStream in) throws IOException {
    if (in == null) {
        throw new IllegalArgumentException("InputStream cannot be null");
    }
    try {
        this.buffer = new byte[0];
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        byte[] buffer = new byte[1024];
        int bytesRead;
        while ((bytesRead = in.read(buffer)) != -1) {
            baos.write(buffer, 0, bytesRead);
        }
        this.buffer = baos.toByteArray();
        this.readPointer = 0;
    } catch (IOException e) {
        throw new IOException("Error reading from InputStream", e);
    }
}
```



Official-Implementation:

```
public void readFrom(final InputStream in) throws IOException {
    pointer = 0;
    size = 0;
    int n;
    do {
        n = in.read(buffer, size, buffer.length - size);
        if (n > 0) {
            size += n;
        }
        resizeIfNeeded();
    } while (n >= 0);
}
```

